COMP3353 Bioinformatics — Assignment 2: Sequence alignment and variant calling

Name: Zheng Choi I
University Number: 3035987788
Email: u3598778@connect.hku.hk

All code is Python 3 (standard library; `scipy` only for Q6) or bash, and is also in the `src/` folder of this submission. Every Python script reads its input path from the command line, as the questions require. `src/fasta.py` is a small FASTA reader that Q2–Q5 import:

```
"""Minimal FASTA reader shared by the Assignment 2 scripts."""


def read_fasta(path):
    """Return a list of (header, sequence) tuples; sequences are
upper-cased."""
    records = []
    header, chunks = None, []
    with open(path) as f:
        for line in f:
            line = line.strip()
            if not line:
                continue
            if line.startswith(">"):
                if header is not None:
                    records.append((header, "".join(chunks).upper()))
                header, chunks = line[1:], []
            else:
                chunks.append(line)
    if header is not None:
        records.append((header, "".join(chunks).upper()))
    return records
```

`bash src/run_all.sh` (run from `assignment2/`) re-runs Q2–Q6 and writes every output to `results/`.

Question 1 — Sequence alignment with bwa (ERR2337147, *S. aureus*)

Input files (in `data/Q1/`): `ref.fna.gz`, the *S. aureus* NCTC 8325 chromosome (NC_007795.1, 2,821,361 bp), and `ERR2337147_1.fastq.gz` / `ERR2337147_2.fastq.gz` (184,803 read pairs of 250 bp, Illumina MiSeq). Tools used: bwa 0.7.18 and samtools 1.24.

1(a) Commands and igv visualisation

Install the tools (same conda environment as tutorial 2):

```
conda create -n comp3353 -c bioconda -c conda-forge bwa samtools igv
conda activate comp3353
```

I ran every step with `src/run_q1.sh` (run from `assignment2/`), and the full output is in `results/q1_output.txt`:

```
#!/usr/bin/env bash
# Q1: index, align, sort, index and inspect ERR2337147 (S. aureus)
with bwa/samtools.
# Usage (from assignment2/):  bash src/run_q1.sh [ref.fa]
[reads_1.fastq.gz] [reads_2.fastq.gz]
# Needs: conda install -c bioconda bwa samtools igv
set -euo pipefail

REF=${1:-data/Q1/ref.fna}
R1=${2:-data/Q1/ERR2337147_1.fastq.gz}
R2=${3:-data/Q1/ERR2337147_2.fastq.gz}
OUT=results/q1
mkdir -p "$OUT"

# unzip the reference if only ref.fna.gz is present (samtools
faidx/igv want plain FASTA)
[[ -s $REF ]] || gunzip -k "$REF.gz"

# (a) index the reference, align the pair, sort and index the
alignments
bwa index "$REF"
samtools faidx "$REF"                                    # lets igv load
the genome
bwa mem -t 4 "$REF" "$R1" "$R2" | samtools view -bS - >
"$OUT/ERR2337147.bam"
samtools sort "$OUT/ERR2337147.bam" -o "$OUT/ERR2337147.sorted.bam"
samtools index "$OUT/ERR2337147.sorted.bam"
samtools flagstat "$OUT/ERR2337147.sorted.bam"

# the pipes below end in `head`, which closes them early; that is not
an error
set +o pipefail

# help pick a well-covered region for the igv screenshot: mean depth
per 1 kb window
samtools depth -a "$OUT/ERR2337147.sorted.bam" \
    | awk '{w=int(($2-1)/1000); s[$1"\t"w*1000+1"-"w*1000+1000]+=$3}
           END{for(k in s) print k"\t"s[k]/1000}' \
    | sort -k3,3gr | head -5
# then:  igv -g "$REF" "$OUT/ERR2337147.sorted.bam"   and go to one of
those regions

# (b) the 10th record of the sorted alignment file and its FLAG
samtools view "$OUT/ERR2337147.sorted.bam" | sed -n 10p
```

```
python3 src/q1_decode_flag.py "$(samtools view
"$OUT/ERR2337147.sorted.bam" | sed -n 10p | cut -f2)"

# (c) reads with MAPQ < 10: CIGAR summary, then examples with MAPQ in
[1, 5]
echo "MAPQ<10 (mapped) records: $(samtools view -F 4
"$OUT/ERR2337147.sorted.bam" | awk '$5<10' | wc -l)"
echo "most common CIGARs among them:"
samtools view -F 4 "$OUT/ERR2337147.sorted.bam" | awk '$5<10 {print
$6}' | sort | uniq -c | sort -k1,1nr | head -15
echo "fraction with soft/hard clipping (S/H):"
samtools view -F 4 "$OUT/ERR2337147.sorted.bam" | awk '$5<10 {n++; if
($6 ~ /[SH]/) c++} END {print c "/" n}'
echo "examples with 1 <= MAPQ <= 5 (QNAME FLAG RNAME POS MAPQ CIGAR +
XA/SA tags):"
samtools view -F 4 "$OUT/ERR2337147.sorted.bam" \
    | awk '$5>=1 && $5<=5 {tags=""; for(i=12;i<=NF;i++) if($i ~ /^(XA|
SA|AS|XS|NM):/) tags=tags" "$i;
                           print $1, $2, $3, $4, $5, $6 tags}' | head
-10
```

The steps:

1. `gunzip -k ref.fna.gz`, then `bwa index ref.fna` builds the FM-index of the reference (`.amb .ann .bwt .pac .sa`). `samtools faidx ref.fna` writes the `.fai` that igv needs.
2. `bwa mem -t 4 ref.fna R1 R2 | samtools view -bS - > ERR2337147.bam` aligns the two FASTQ files as pairs and saves the output as BAM.
3. `samtools sort … -o ERR2337147.sorted.bam` orders the records by coordinate. `samtools index` writes the `.bai` used for region lookups and by igv.
4. `igv -g data/Q1/ref.fna results/q1/ERR2337147.sorted.bam` loads the genome and the alignments. I then typed the region below into the locus box.

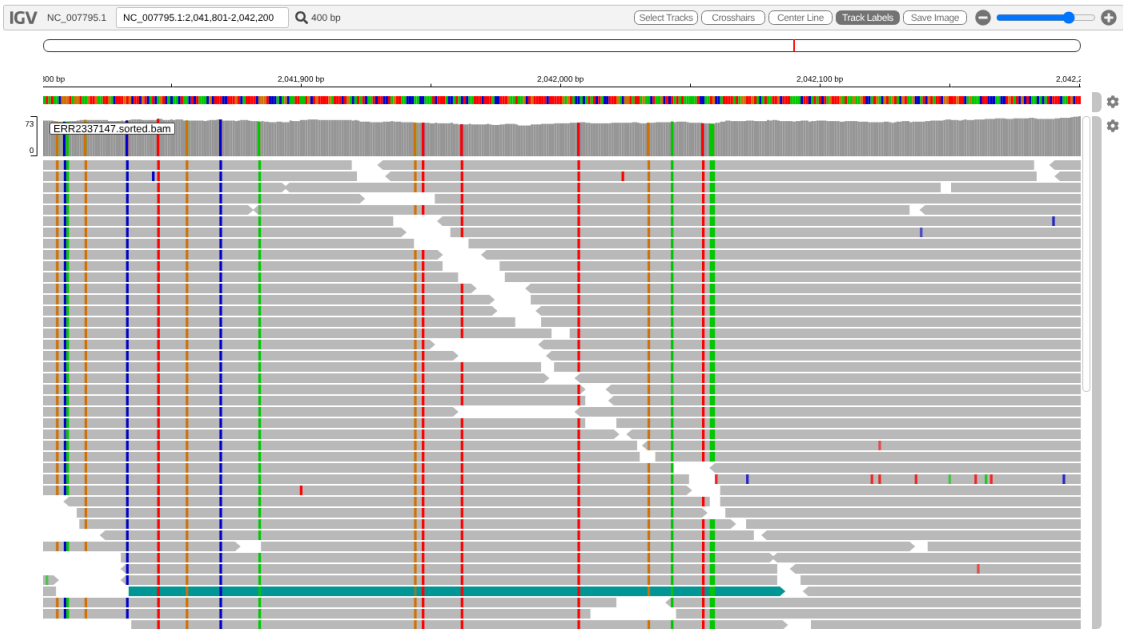
`samtools flagstat` on the sorted BAM:

```
374323 + 0 in total (QC-passed reads + QC-failed reads)
369606 + 0 primary
0 + 0 secondary
4717 + 0 supplementary
0 + 0 duplicates
0 + 0 primary duplicates
355435 + 0 mapped (94.95% : N/A)
350718 + 0 primary mapped (94.89% : N/A)
369606 + 0 paired in sequencing
184803 + 0 read1
184803 + 0 read2
345552 + 0 properly paired (93.49% : N/A)
348908 + 0 with itself and mate mapped
1810 + 0 singletons (0.49% : N/A)
```

```
0 + 0 with mate mapped to a different chr
0 + 0 with mate mapped to a different chr (mapQ>=5)
```

94.9% of the reads mapped and 93.5% are properly paired. Mean depth over the whole genome is 30.2× (`samtools depth -a`). bwa estimated the insert size at 364 ± 80 bp (FR orientation).

Region shown: `NC_007795.1:2,041,801-2,042,200` (400 bp). I wanted a region with good coverage and no ambiguous mappings. The 2 kb window `NC_007795.1:2,041,001-2,043,000` has mean depth 65.5× (min 43×, max 81×). It contains 579 reads, 577 of them with MAPQ 60, and no read with MAPQ < 10.



igv screenshot of NC_007795.1:2,041,801-2,042,200

(This screenshot is from igv.js 3.1.2, the web version of IGV, showing the same sorted BAM and reference; `tools/igv_snapshot/` has the page used. Desktop IGV shows the same view after `igv -g data/Q1/ref.fna results/q1/ERR2337147.sorted.bam` with `NC_007795.1:2,041,801-2,042,200` typed in the locus box.)

The top track is the reference sequence. Below it are the coverage histogram (about 70×) and the piled-up reads. Each grey bar is one read, and the arrow end shows its strand. Coloured ticks are bases that differ from the reference. Several columns are coloured in *every* read covering them. Those are not sequencing errors. They are real differences (SNPs) between the sequenced isolate and the NCTC 8325 reference strain. Isolated single ticks in one read are sequencing errors.

1(b) FLAG of the 10th record

```
$ samtools view results/q1/ERR2337147.sorted.bam | sed -n 10p | cut -
f1-9
ERR2337147.11   163 NC_007795.1 1   60  97S153M =   105 352
```

Decoded with `src/q1_decode_flag.py`:

```python
"""Q1b helper: decode a SAM FLAG into its individual bits.

Usage:
    python q1_decode_flag.py <flag>
    samtools view sorted.bam | sed -n 10p | cut -f2 | xargs python
q1_decode_flag.py
"""
import sys

FLAG_BITS = [
    (0x1, "PAIRED", "template has multiple segments (read is
paired)"),
    (0x2, "PROPER_PAIR", "each segment properly aligned (proper
pair)"),
    (0x4, "UNMAP", "this read is unmapped"),
    (0x8, "MUNMAP", "the mate (next segment) is unmapped"),
    (0x10, "REVERSE", "this read is mapped to the reverse strand"),
    (0x20, "MREVERSE", "the mate is mapped to the reverse strand"),
    (0x40, "READ1", "this is the first read in the pair"),
    (0x80, "READ2", "this is the second read in the pair"),
    (0x100, "SECONDARY", "secondary (non-primary) alignment"),
    (0x200, "QCFAIL", "read fails platform/vendor quality checks"),
    (0x400, "DUP", "PCR or optical duplicate"),
    (0x800, "SUPPLEMENTARY", "supplementary (chimeric) alignment"),
]


def decode(flag):
    print(f"FLAG = {flag} = 0b{flag:012b}")
    for bit, name, meaning in FLAG_BITS:
        state = "SET  " if flag & bit else "unset"
        print(f"  0x{bit:03x} ({bit:4d}) {state} {name:<13}
{meaning}")
    if flag & 0x1:
        which = "first" if flag & 0x40 else "second" if flag & 0x80
else "unknown"
        print(f"=> {which} read in the pair; mate is "
              f"{'UNMAPPED' if flag & 0x8 else 'mapped'}")


if __name__ == "__main__":
    if len(sys.argv) != 2:
        sys.exit("Usage: python q1_decode_flag.py <flag>")
    decode(int(sys.argv[1], 0))
```

```
FLAG = 163 = 0b000010100011
  0x001 (   1) SET   PAIRED        template has multiple segments
(read is paired)
  0x002 (   2) SET   PROPER_PAIR   each segment properly aligned
(proper pair)
```

```
  0x004 (   4) unset UNMAP         this read is unmapped
  0x008 (   8) unset MUNMAP        the mate (next segment) is unmapped
  0x010 (  16) unset REVERSE       this read is mapped to the reverse
strand
  0x020 (  32) SET   MREVERSE      the mate is mapped to the reverse
strand
  0x040 (  64) unset READ1         this is the first read in the pair
  0x080 ( 128) SET   READ2         this is the second read in the pair
  0x100 ( 256) unset SECONDARY     secondary (non-primary) alignment
  0x200 ( 512) unset QCFAIL        read fails platform/vendor quality
checks
  0x400 (1024) unset DUP           PCR or optical duplicate
  0x800 (2048) unset SUPPLEMENTARY supplementary (chimeric) alignment
=> second read in the pair; mate is mapped
```

FLAG = 163 = 128 + 32 + 2 + 1 (binary `0000 1010 0011`):

bit (hex)	decimal	name	set?	meaning for this read
0x1	1	PAIRED	**yes**	the read comes from a paired-end template
0x2	2	PROPER_PAIR	**yes**	both mates aligned in the expected FR orientation at a plausible distance (TLEN 352, close to the 364 bp mean insert)
0x4	4	UNMAP	no	the read itself is mapped
0x8	8	MUNMAP	no	the mate is mapped
0x10	16	REVERSE	no	the read aligned to the forward strand
0x20	32	MREVERSE	**yes**	the mate aligned to the reverse strand
0x40	64	READ1	no	not the first read
0x80	128	READ2	**yes**	the second read of the pair (from `ERR2337147_2.fastq.gz`)

bit (hex)	decimal	name	set?	meaning for this read
0x100	256	SECONDARY	no	primary alignment
0x200	512	QCFAIL	no	passed QC
0x400	1024	DUP	no	not marked as a duplicate
0x800	2048	SUPPLEMENTARY	no	not a supplementary (split) alignment

Answer: this is the **second read** of the pair (0x80 set, 0x40 not set), and its **mate is mapped** (0x8 not set). The other fields agree: RNEXT =, PNEXT 105 (the mate starts at position 105), and the pair is properly paired on opposite strands.

Its CIGAR `97S153M` and POS 1 are also worth noting. The read starts at the very first base of the chromosome, and its first 97 bases are soft-clipped. The *S. aureus* chromosome is circular, so those 97 bases come from the end of the linear reference sequence, which the aligner cannot wrap around to.

1(c) Reads with MAPQ < 10

```
MAPQ<10 (mapped) records: 3269
most common CIGARs among them:
    328 251M
    160 250M
     33 249M
     10 248M
      9 65M186H
      7 47M204H
      7 51M200H
      6 185H66M
      6 187H64M
      6 195H56M
      6 196H55M
      6 199H52M
      6 201H50M
      6 202H49M
      6 209H42M
fraction with soft/hard clipping (S/H):
2602/3269
examples with 1 <= MAPQ <= 5 (QNAME FLAG RNAME POS MAPQ CIGAR + XA/SA
tags):
ERR2337147.4930 73 NC_007795.1 36501 2 8M3I25M1I14M3D69M131S NM:i:13
AS:i:62 XS:i:59 XA:Z:NC_007795.1,+42681,51S69M131S,2;
ERR2337147.4937 73 NC_007795.1 36504 2 4M1D25M1I14M3D69M138S NM:i:11
AS:i:62 XS:i:59 XA:Z:NC_007795.1,+42681,44S69M138S,2;
ERR2337147.5185 185 NC_007795.1 36551 5 9S240M NM:i:6 AS:i:210
```

```
XS:i:205 XA:Z:NC_007795.1,-42681,9S240M,7;
ERR2337147.5193 121 NC_007795.1 42772 5 250M NM:i:7 AS:i:215 XS:i:210
XA:Z:NC_007795.1,-36642,250M,8;
```

All 10 examples are in `results/q1_output.txt`. More counts:

MAPQ	0	1	2	3	4	5	6	7	8	9
records	1937	118	161	152	117	165	163	167	94	195

Of the 355,435 mapped records, 3,269 (0.9%) have MAPQ < 10. Their CIGAR strings fall into two groups:

1. **Clipped alignments: 2,602 of 3,269 (80%)** have soft (`S`) or hard (`H`) clipping, and many also have indels (`I`/`D`). Only part of the read aligns. 1,850 of them carry an `SA:Z` tag, so the read is split across two places (chimeric or spanning a structural difference from the reference). The **…M…H** CIGARs such as `65M186H` and `47M204H` in the list above are the short supplementary pieces of those split reads; bwa hard-clips the rest of the read in supplementary records. A short aligned piece scores low, so its score is close to the next-best hit, and MAPQ drops.
2. **Full-length matches: 585 records** are plain `M` with no clipping and no indels (`251M`, `250M`, `249M` …). The whole read aligns, but 574 of them have an `XA:Z` tag listing another equally good or nearly equally good location, and 465 have MAPQ 0. These reads come from **repeats**: rRNA operons, insertion sequences/transposons and duplicated genes, where the aligner cannot tell which copy the read came from. MAPQ is the aligner's confidence in the *position* of a read, not a measure of how well the bases match. So a perfect `250M` alignment can still get MAPQ 0.

In short: low MAPQ comes either from a read that only partly aligns (clipped CIGAR) or from a read that aligns equally well in more than one place (repeat; full-length `M` plus an `XA` tag).

Example read with MAPQ in [1, 5]:

```
ERR2337147.5185  FLAG 185  NC_007795.1:36551  MAPQ 5  CIGAR 9S240M
NM:i:6  AS:i:210  XS:i:205  XA:Z:NC_007795.1,-42681,9S240M,7;
```

- `9S` means the first 9 bases of the read are **soft-clipped**. They stay in SEQ but are not aligned, most likely low-quality bases or bases that do not match the reference here.
- `240M` means the next 240 bases align to the reference starting at POS 36,551. `M` covers matches *and* mismatches. `NM:i:6` says 6 of the 240 bases differ from the reference, and there are no insertions or deletions.
- The read is 9 + 240 = 249 bp long and covers reference positions 36,551–36,790.

The read aligns in full, so the CIGAR alone does not explain the low MAPQ. The tags do. The best alignment scores `AS = 210`, and the second-best hit scores `XS = 205`. That second hit is listed in `XA`: position 42,681 on the reverse strand, with the *same* CIGAR `9S240M` and 7 mismatches instead of 6. This stretch of sequence occurs twice in the genome, about 6 kb apart, and the read fits both copies almost equally well; the best copy wins by just one mismatch. bwa therefore

gives it the primary position but only MAPQ 5, about a 30% chance (10^(−5/10) ≈ 0.32) that the position is wrong. FLAG 185 also shows that its mate is unmapped (0x8), so there is no proper pair to help pick the right copy.

Question 2 — Hamming distance

Source code (`src/q2_hamming.py`)

```
"""Q2: Hamming distance between the two equal-length sequences in a
FASTA file.

Usage:
    python q2_hamming.py <fasta_path>
"""
import sys

from fasta import read_fasta


def hamming(s, t):
    if len(s) != len(t):
        raise ValueError(f"sequences differ in length ({len(s)} vs
{len(t)})")
    return sum(a != b for a, b in zip(s, t))


if __name__ == "__main__":
    if len(sys.argv) != 2:
        sys.exit("Usage: python q2_hamming.py <fasta_path>")
    (_, s), (_, t) = read_fasta(sys.argv[1])[:2]
    print(hamming(s, t))
```

The script compares the two sequences position by position and counts the mismatches. It raises an error if the lengths differ. Check: the Rosalind example (`GAGCCTACTAACGGGAT` / `CATCGTAATGACGGCCT`) gives 7.

Output on `data/Q2/input.fa`

```
$ python3 src/q2_hamming.py data/Q2/input.fa
79
```

Answer: the Hamming distance is 79. Both sequences (`Sequence_1`, `Sequence_2`) are 500 bp long, and they differ at 79 of the 500 positions.

Question 3 — Edit distance

Source code (`src/q3_edit_distance.py`)

```
"""Q3: Edit (Levenshtein) distance between the two sequences in a
FASTA file.

Usage:
    python q3_edit_distance.py <fasta_path>

D[i][j] = edit distance between s[:i] and t[:j]
D[0][0] = 0, D[i][0] = i, D[0][j] = j
D[i][j] = min(D[i-1][j] + 1,                      # delete s[i-1]
              D[i][j-1] + 1,                      # insert t[j-1]
              D[i-1][j-1] + (s[i-1] != t[j-1]))  # match / substitute
"""
import sys

from fasta import read_fasta


def edit_distance(s, t):
    n, m = len(s), len(t)
    # Only the previous row is needed to compute the current one, so
the
    # (n+1) x (m+1) matrix is filled row by row with O(m) memory.
    prev = list(range(m + 1))                   # row i = 0: D[0][j] = j
    for i in range(1, n + 1):
        cur = [i] + [0] * m                     # D[i][0] = i
        si = s[i - 1]
        for j in range(1, m + 1):
            cur[j] = min(prev[j] + 1,
                         cur[j - 1] + 1,
                         prev[j - 1] + (si != t[j - 1]))
        prev = cur
    return prev[m]


if __name__ == "__main__":
    if len(sys.argv) != 2:
        sys.exit("Usage: python q3_edit_distance.py <fasta_path>")
    (_, s), (_, t) = read_fasta(sys.argv[1])[:2]
    print(edit_distance(s, t))
```

How the distance matrix is populated

`D` has `len(s)+1` rows and `len(t)+1` columns. `D(i, j)` is the edit distance between the first `i` symbols of `s` and the first `j` symbols of `t`.

1. **Initial values.** `D(0,0) = 0`. `D(i,0) = i`: turning `s[:i]` into the empty string takes `i` deletions. `D(0,j) = j`: building `t[:j]` from nothing takes `j` insertions.

2. **Recurrence.** I fill row by row, left to right. That way, when I compute cell `(i, j)`, the three cells it depends on (up, left and up-left) are already filled:

```
D(i,j) = min( D(i-1,j)   + 1,                  # delete s_i
              D(i,j-1)   + 1,                  # insert t_j
              D(i-1,j-1) + [s_i != t_j] )      # match (cost 0)
or substitute (cost 1)
```

   The optimal alignment of `s[:i]` and `t[:j]` must end in one of these three ways, so the smallest of the three costs is the answer for that cell.

3. **Result.** The edit distance is `D(len(s), len(t))`, the bottom-right cell.

Row `i` only depends on row `i-1`, so the code keeps just two rows in memory. Run time is still O(|s|·|t|), but memory drops to O(|t|). That matters for long input sequences. Check: `PLEASANTLY/MEANLY` gives 5, the textbook value.

Output

```
$ python3 src/q3_edit_distance.py data/Q2/input.fa
7
$ python3 src/q3_edit_distance.py data/Q3/input.fa
1704
```

Answer for `data/Q3/input.fa`: the edit distance is 1704. The file holds two hepatitis B virus genomes: X02763.1 (serotype adw2, 3,221 bp) and NC_003977.2 (strain ayw, 3,182 bp). Computing it fills a 3,222 × 3,183 matrix, about 10 million cells, which takes about 1.5 s.

1704 is much higher than the true difference between the two viruses. The HBV genome is circular, and the two records start at different points of the circle: the first 30 bases of X02763.1 appear at position 1,815 of NC_003977.2. Edit distance compares the strings as linear sequences, so it has to delete a large block at one end and insert it again at the other end. When I rotate NC_003977.2 so that both start at the same base, the edit distance drops to **335**, about 10% of the genome. That is roughly what you would expect between two different HBV strains. The 39 bp length difference (3,221 vs 3,182) is part of those 335 edits.

Comparison with Q2: On `data/Q2/input.fa` the edit distance is **7**, but the Hamming distance is **79**. The Hamming distance is one valid way to edit `s` into `t`: substitutions only, no gaps. So the edit distance can never be larger (`edit ≤ Hamming`). Here it is much smaller because the two sequences differ by a few insertions and deletions, which Hamming distance cannot model. One optimal set of 7 edits (positions in `Sequence_1`, 1-based):

position in s	edit	cost
159	substitute C → G	1
after 200	insert GC	2
301–302	delete AA	2
435	substitute C → A	1
440	substitute A → G	1
	total	**7**

The 2-base insertion after position 200 shifts every later base of `t` two places to the right. The 2-base deletion at 301–302 shifts them back. Between the two, about 100 positions are compared out of register. Hamming distance counts most of them as mismatches: 79 = the 3 real substitutions plus 76 mismatches caused by the shift. Edit distance aligns around the gap and charges only 2 + 2 for the indels.

Question 4 — BWT encoder and decoder

Source code (`src/bwt.py`, encoder and decoder)

```
"""Q4: Burrows-Wheeler Transform encoder and decoder ('$' is the end
marker).

Usage:
    python bwt.py encode <fasta_path>      # prints the BWT of the
sequence
    python bwt.py decode <bwt_txt_path>    # prints the original
sequence
"""
import sys

from fasta import read_fasta

SENTINEL = "$"


def suffix_array(text):
    """Suffix array by prefix doubling: O(n log^2 n), fine for whole
genomes."""
    n = len(text)
    rank = [ord(c) for c in text]
    sa = list(range(n))
    k = 1
    while True:
        key = lambda i: (rank[i], rank[i + k] if i + k < n else -1)
        sa.sort(key=key)
        new_rank = [0] * n
        for a, b in zip(sa, sa[1:]):
            new_rank[b] = new_rank[a] + (key(a) != key(b))
        rank = new_rank
        if rank[sa[-1]] == n - 1:
            return sa
        k *= 2


def bwt_encode(seq):
    """BWT = last column of the sorted rotations of seq + '$'.
```

```python
    Sorting the rotations of a '$'-terminated string is the same as
sorting
    its suffixes, and the last-column symbol of rotation i is text[i-
1].
    """
    text = seq + SENTINEL
    return "".join(text[i - 1] for i in suffix_array(text))


def bwt_decode(bwt):
    """Invert the BWT with the LF mapping (last-to-first column)."""
    # first[c] = row of the first occurrence of c in the sorted first
column F
    counts = {}
    for c in bwt:
        counts[c] = counts.get(c, 0) + 1
    first, total = {}, 0
    for c in sorted(counts):
        first[c] = total
        total += counts[c]
    # LF[i] = row in F holding the same character occurrence as L[i]
    seen, lf = {}, [0] * len(bwt)
    for i, c in enumerate(bwt):
        lf[i] = first[c] + seen.get(c, 0)
        seen[c] = seen.get(c, 0) + 1
    # Row 0 is the rotation starting with '$'; its last char is the
final
    # symbol of the sequence.  Walk LF backwards through the original
text.
    out, row = [], 0
    for _ in range(len(bwt) - 1):
        out.append(bwt[row])
        row = lf[row]
    return "".join(reversed(out))


if __name__ == "__main__":
    if len(sys.argv) != 3 or sys.argv[1] not in ("encode", "decode"):
        sys.exit("Usage: python bwt.py encode <fasta_path> | decode
<bwt_txt_path>")
    mode, path = sys.argv[1], sys.argv[2]
    if mode == "encode":
        print(bwt_encode(read_fasta(path)[0][1]))
    else:
        with open(path) as f:
            print(bwt_decode("".join(f.read().split())))
```

4(a) Encoder

I append `$` (sorts before A, C, G, T) to the sequence. The BWT is the last column of the sorted list of all rotations. Sorting rotations of a `$`-terminated string is the same as sorting its suffixes, so the code builds a suffix array by prefix doubling. That needs O(n) memory, not the n^2 of a full rotation matrix, so whole-genome inputs work. The last-column character of the rotation starting at `i` is `text[i-1]`. Check: `BANANA` → `ANNB$AA`. Random DNA strings match a brute-force sort of all rotations.

```
$ python3 src/bwt.py encode data/Q4/input1.fa
TC$GAATCCGG
$ python3 src/bwt.py encode data/Q4/input2.fa | cut -c1-50
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAACTGTCTTCTTTCCTTGG
```

input1 (`ACGTGCACGT`, 10 bp). The sorted rotations of `ACGTGCACGT$` are:

row	rotation	last column
0	`$ACGTGCACGT`	T
1	`ACGT$ACGTGC`	C
2	`ACGTGCACGT$`	$
3	`CACGT$ACGTG`	G
4	`CGT$ACGTGCA`	A
5	`CGTGCACGT$A`	A
6	`GCACGT$ACGT`	T
7	`GT$ACGTGCAC`	C
8	`GTGCACGT$AC`	C
9	`T$ACGTGCACG`	G
10	`TGCACGT$ACG`	G

Reading the last column top to bottom gives **BWT = `TC$GAATCCGG`**.

input2 is the SARS-CoV-2 reference genome (NC_045512.2, 29,903 bp). The first 50 BWT symbols are `AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAACTGTCTTCTTTCCTTGG`.

What I find: the BWT starts with a run of **33 A's**. The genome ends with a 33-nt poly-A tail (**…** `GAATGAC` followed by 33 `A`). In the sorted rotation matrix, row 0 is the rotation starting with `$`, and its last character is the final base of the genome, an A. The rows right after it start with `A$`, `AA$`, `AAA$`, …, i.e. the suffixes inside the poly-A tail. Each of those is preceded by another A, so all of them put an A in the last column. Row 33 (`AAA…A$`, the whole tail) is preceded by the `C` before the tail, which is where the run ends.

This is the general property of the BWT: rotations that start with the same context sort next to each other, and the same context tends to be preceded by the same base. So the BWT groups identical symbols into runs. Over the whole input2 BWT there are 21,518 runs for 29,904 symbols (longest run 33). This is why the BWT compresses well with run-length encoding. The

BWT holds exactly the same symbols as the input plus one `$`, only reordered, which is what makes it reversible (4b) and searchable (Q5).

4(b) Decoder

The decoder uses the **LF mapping**. The first column `F` is `sorted(L)`. The k-th occurrence of character `c` in `L` and the k-th occurrence of `c` in `F` are the same character of the text. So `LF(i) = C[c] + (number of c in L[:i])`. Row 0 of the sorted rotations starts with `$`, so `L[0]` is the last base of the sequence. Applying `row = LF(row)` repeatedly walks the text backwards, one base per step, and reversing the collected bases gives the original. It runs in O(n).

```
$ python3 src/bwt.py decode data/Q4/input3.txt
AGTTGTTGATCTGTGTGAGTCAGACTGCGACAGTTCGAGTCTGAAGCGAGA
$ python3 src/bwt.py encode data/Q4/input1.fa > results/input1.bwt.txt
$ python3 src/bwt.py decode results/input1.bwt.txt
ACGTGCACGT
```

input3: the BWT string `AGGGGGCAGGC$GTAGGTTATATCACTCTTAAATTATAGTGACGTCGCTGGG` (52 symbols) decodes to the 51 bp sequence **`AGTTGTTGATCTGTGTGAGTCAGACTGCGACAGTTCGAGTCTGAAGCGAGA`**. Encoding that sequence again gives back exactly the input3 string.

Is it the same as the original? Yes. Decoding `TC$GAATCCGG` gives `ACGTGCACGT`, which is identical to the sequence in `data/Q4/input1.fa` (`run_all.sh` prints `identical: True`). The BWT can be inverted exactly because the `$` marker fixes where the text ends. I also tested encode→decode on 300 random strings and one random 200 kb string, and every one came back unchanged.

Question 5 — Search with BWT (FM-index)

Source code (`src/q5_fm_search.py`, reuses `bwt_encode` from Q4)

```python
"""Q5: count occurrences of a motif with FM-index backward search.

Usage:
    python q5_fm_search.py <fasta_path> <motif> [--show-occ N]

Prints the BWT, the count array C and the occurrence matrix Occ (the
full
matrix for short inputs, otherwise its first N rows), then the
backward-search
steps and the number of occurrences.
"""
import sys

from bwt import bwt_encode
from fasta import read_fasta
```

```
def build_fm_index(bwt):
    alphabet = sorted(set(bwt))
    # C[c] = number of symbols in the text lexicographically smaller
than c
    C, total = {}, 0
    for c in alphabet:
        C[c] = total
        total += bwt.count(c)
    # Occ[c][k] = occurrences of c in bwt[:k]   (k = 0 .. n)
    occ = {c: [0] * (len(bwt) + 1) for c in alphabet}
    for k, ch in enumerate(bwt):
        for c in alphabet:
            occ[c][k + 1] = occ[c][k] + (ch == c)
    return C, occ


def backward_search(motif, C, occ, n, trace=None):
    """Return the number of occurrences; SA interval is [lo, hi)."""
    lo, hi = 0, n
    for c in reversed(motif):
        if c not in C:
            return 0
        lo, hi = C[c] + occ[c][lo], C[c] + occ[c][hi]
        if trace is not None:
            trace.append((c, lo, hi))
        if lo >= hi:
            return 0
    return hi - lo


def main():
    args = sys.argv[1:]
    show = 30
    if "--show-occ" in args:
        i = args.index("--show-occ")
        show = int(args[i + 1])
        del args[i:i + 2]
    if len(args) != 2:
        sys.exit("Usage: python q5_fm_search.py <fasta_path> <motif>
[--show-occ N]")
    path, motif = args[0], args[1].upper()

    bwt = bwt_encode(read_fasta(path)[0][1])
    n = len(bwt)
    C, occ = build_fm_index(bwt)
    alphabet = sorted(C)

    print(f"BWT (length {n}):")
    print(bwt if n <= 200 else bwt[:200] + "...")
```

```
    print("\nCount array C:")
    print("   " + "   ".join(f"C[{c}]={C[c]}" for c in alphabet))
    rows = n + 1 if n + 1 <= show else show
    print(f"\nOccurrence matrix Occ(c, k), k = 0..{n}"
          + ("" if rows == n + 1 else f" (first {rows} rows shown)") +
":")
    print("    k  L[k-1]  " + "   ".join(f"{c:>6}" for c in alphabet))
    for k in range(rows):
        prev = bwt[k - 1] if k else "-"
        print(f"{k:4d}  {prev:>6}  " + "   ".join(f"{occ[c][k]:6d}" for
c in alphabet))
    print(f"  (final row k={n}: " + ", ".join(f"{c}={occ[c][n]}" for c
in alphabet) + ")")

    trace = []
    count = backward_search(motif, C, occ, n, trace)
    print(f"\nBackward search for {motif}: start with [lo, hi) = [0,
{n})")
    for c, lo, hi in trace:
        print(f"  char {c}: lo = C[{c}] + Occ({c},lo), hi = C[{c}] +
Occ({c},hi)"
              f" -> [{lo}, {hi})  size {max(hi - lo, 0)}")
    print(f"\nNumber of occurrences of {motif}: {count}")


if __name__ == "__main__":
    main()
```

Method

- `C[c]` = number of symbols in `t$` that are smaller than `c` (`$ < A < C < G < T`). It is the row where the rows starting with `c` begin in the sorted rotation matrix.
- `Occ(c, k)` = number of times `c` appears in `t'[0..k-1]`, the first `k` symbols of the BWT. It is built as cumulative counts with `n+1` rows.
- **Backward search.** Start with the interval of all rows `[lo, hi) = [0, n)` and process the motif from its **last** character to its first: `lo ← C[c] + Occ(c, lo)`, `hi ← C[c] + Occ(c, hi)`. After every step, `[lo, hi)` holds exactly the rows whose rotations start with the suffix of the motif read so far. When the whole motif is processed, the number of occurrences is `hi − lo`. If the interval becomes empty, the count is 0. Each character costs O(1), so a search takes O(|s|) whatever the size of the text.

Check: the counts match a brute-force overlapping regex count on 300 random strings. On `ACGTACGTTACGTAACG` the trace is `T:[14,18) → G:[11,14) → C:[7,10) → A:[3,6)`, which gives 3.

Output on {data/Q4/input1.fa, s = "ACGT"} (BWT, C, Occ and search steps)

```
$ python3 q5_fm_search.py ../data/Q4/input1.fa ACGT --show-occ 1000
BWT (length 11):
TC$GAATCCGG
```

```
Count array C:
  C[$]=0  C[A]=1  C[C]=3  C[G]=6  C[T]=9

Occurrence matrix Occ(c, k), k = 0..11:
   k  L[k-1]       $       A       C       G       T
   0       -       0       0       0       0       0
   1       T       0       0       0       0       1
   2       C       0       0       1       0       1
   3       $       1       0       1       0       1
   4       G       1       0       1       1       1
   5       A       1       1       1       1       1
   6       A       1       2       1       1       1
   7       T       1       2       1       1       2
   8       C       1       2       2       1       2
   9       C       1       2       3       1       2
  10       G       1       2       3       2       2
  11       G       1       2       3       3       2
  (final row k=11: $=1, A=2, C=3, G=3, T=2)

Backward search for ACGT: start with [lo, hi) = [0, 11)
  char T: lo = C[T] + Occ(T,lo), hi = C[T] + Occ(T,hi) -> [9, 11)
size 2
  char G: lo = C[G] + Occ(G,lo), hi = C[G] + Occ(G,hi) -> [7, 9)  size
2
  char C: lo = C[C] + Occ(C,lo), hi = C[C] + Occ(C,hi) -> [4, 6)  size
2
  char A: lo = C[A] + Occ(A,lo), hi = C[A] + Occ(A,hi) -> [1, 3)  size
2

Number of occurrences of ACGT: 2
```

Answer: `ACGT` occurs 2 times in input1. By hand, the steps are:

1. Start with all rows `[0, 11)`.
2. T: `lo = C[T] + Occ(T,0) = 9 + 0 = 9`, `hi = C[T] + Occ(T,11) = 9 + 2 = 11` → `[9, 11)`, the rows starting with `T`.
3. G: `lo = 6 + Occ(G,9) = 6 + 1 = 7`, `hi = 6 + Occ(G,11) = 6 + 3 = 9` → `[7, 9)`, the rows starting with `GT`.
4. C: `lo = 3 + Occ(C,7) = 3 + 1 = 4`, `hi = 3 + Occ(C,9) = 3 + 3 = 6` → `[4, 6)`, the rows starting with `CGT`.
5. A: `lo = 1 + Occ(A,4) = 1 + 0 = 1`, `hi = 1 + Occ(A,6) = 1 + 2 = 3` → `[1, 3)`, the rows starting with `ACGT`.

`hi − lo = 3 − 1 = 2`. Rows 1 and 2 in the rotation table of Q4(a) are `ACGT$ACGTGC` and `ACGTGCACGT$`, i.e. the matches at positions 7 and 1 of `ACGTGCACGT`.

Output on {data/Q4/input2.fa, s = "ACGT"}

```
$ python3 q5_fm_search.py ../data/Q4/input2.fa ACGT
BWT (length 29904):
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAACTGTCTTCTTTCCTTGGTTCCTCCCTGGTTGTTGACT
TTTCTTGTTTCTGGATTCCTGAAGTCTGGCTGAACATTCTAGAAGGCTTCTTCGGAGGTTCAAGGAAATC
GGATCGTGAGCCTCAGTGATATTTTCACAGCACCGGGTGCAGACAAGAAACATAACGCCA...

Count array C:
  C[$]=0  C[A]=1  C[C]=8955  C[G]=14447  C[T]=20310

Occurrence matrix Occ(c, k), k = 0..29904 (first 30 rows shown):
   k  L[k-1]       $       A       C       G       T
   0       -       0       0       0       0       0
   1       A       0       1       0       0       0
   2       A       0       2       0       0       0
   3       A       0       3       0       0       0
   4       A       0       4       0       0       0
   ...   (rows 5-29 are all A as well; full output in
results/q5_output.txt)
  (final row k=29904: $=1, A=8954, C=5492, G=5863, T=9594)

Backward search for ACGT: start with [lo, hi) = [0, 29904)
  char T: lo = C[T] + Occ(T,lo), hi = C[T] + Occ(T,hi) -> [20310,
29904)  size 9594
  char G: lo = C[G] + Occ(G,lo), hi = C[G] + Occ(G,hi) -> [18320,
20310)  size 1990
  char C: lo = C[C] + Occ(C,lo), hi = C[C] + Occ(C,hi) -> [12195,
12366)  size 171
  char A: lo = C[A] + Occ(A,lo), hi = C[A] + Occ(A,hi) -> [4167, 4231)
size 64

Number of occurrences of ACGT: 64
```

(For the long input2 the script prints only the BWT's first 200 symbols and the first 30 rows of the `Occ` matrix plus its final row. The full matrix has 29,905 rows.)

Answer: `ACGT` occurs 64 times in the SARS-CoV-2 genome. A direct scan of the sequence for overlapping matches also gives 64.

Question 6 — Variant calling with Bayes' theorem

Model

The reference base is A, and a read reports T. Each read is independent and has a 1% misclassification rate e. The probability that one read shows T under each genotype G:

G	$P(\text{read} = T \mid G)$
AA	$e = 0.01$ (an A misread as T)

G	P(read = T | G)
AT	½(1−e) + ½e = 0.5
TT	1 − e = 0.99

The number of T reads k out of n follows a binomial distribution: `P(D | G) = C(n,k) p_G^k (1−p_G)^(n−k)`. By Bayes' theorem:

```
P(G | D) = P(D | G) P(G) / Σ_G' P(D | G') P(G'),     P(mutant | D) = P(AT | D) + P(TT | D)
```

Prior for AA = 1 − P(AT) − P(TT).

Source code (`src/q6_variant_calling.py`)

```
"""Q6: Bayesian genotype posteriors for a site with reference base A.

P(G | k T-reads of n) = P(k | n, p_G) P(G) / sum_G' P(k | n, p_G') P(G')
with p_G = probability that one read shows T given genotype G:
    AA: e            (an A misread as T)
    AT: 0.5 * (1 - e) + 0.5 * e = 0.5
    TT: 1 - e
"""
from scipy.stats import binom

ERR = 0.01


def posteriors(n_t, n, prior_het, prior_hom, err=ERR):
    p_t = {"AA": err, "AT": 0.5 * (1 - err) + 0.5 * err, "TT": 1 - err}
    prior = {"AA": 1 - prior_het - prior_hom, "AT": prior_het, "TT": prior_hom}
    joint = {g: binom.pmf(n_t, n, p_t[g]) * prior[g] for g in p_t}
    evidence = sum(joint.values())
    print(f"{n - n_t} A / {n_t} T out of {n} reads, "
          f"priors AA={prior['AA']:.5f} AT={prior_het} TT={prior_hom}")
    print(f"  {'G':<3} {'P(T|G)':>7} {'likelihood':>12} {'prior':>8} "
          f"{'lik x prior':>12} {'posterior':>10}")
    for g in p_t:
        lik = binom.pmf(n_t, n, p_t[g])
        print(f"  {g:<3} {p_t[g]:>7.3f} {lik:>12.4e} {prior[g]:>8.5f} "
              f"{joint[g]:>12.4e} {joint[g] / evidence:>10.6f}")
    mutant = (joint["AT"] + joint["TT"]) / evidence
    print(f"  evidence P(D) = {evidence:.4e}")
    print(f"  P(mutant | D) = P(AT|D) + P(TT|D) = {mutant:.6f}\n")
    return mutant
```

```python
if __name__ == "__main__":
    print("(a)"); posteriors(1, 5, 0.02, 0.001)
    print("(b)"); posteriors(6, 30, 0.02, 0.001)
    print("(c)"); posteriors(6, 30, 0.0005, 0.00001)
```

Output

```
$ python3 q6_variant_calling.py
(a)
4 A / 1 T out of 5 reads, priors AA=0.97900 AT=0.02 TT=0.001
  G    P(T|G)   likelihood    prior  lik x prior  posterior
  AA    0.010   4.8030e-02  0.97900   4.7021e-02   0.937682
  AT    0.500   1.5625e-01  0.02000   3.1250e-03   0.062318
  TT    0.990   4.9500e-08  0.00100   4.9500e-11   0.000000
  evidence P(D) = 5.0146e-02
  P(mutant | D) = P(AT|D) + P(TT|D) = 0.062318

(b)
24 A / 6 T out of 30 reads, priors AA=0.97900 AT=0.02 TT=0.001
  G    P(T|G)   likelihood    prior  lik x prior  posterior
  AA    0.010   4.6652e-07  0.97900   4.5672e-07   0.039657
  AT    0.500   5.5300e-04  0.02000   1.1060e-05   0.960343
  TT    0.990   5.5903e-43  0.00100   5.5903e-46   0.000000
  evidence P(D) = 1.1517e-05
  P(mutant | D) = P(AT|D) + P(TT|D) = 0.960343

(c)
24 A / 6 T out of 30 reads, priors AA=0.99949 AT=0.0005 TT=1e-05
  G    P(T|G)   likelihood    prior  lik x prior  posterior
  AA    0.010   4.6652e-07  0.99949   4.6628e-07   0.627751
  AT    0.500   5.5300e-04  0.00050   2.7650e-07   0.372249
  TT    0.990   5.5903e-43  0.00001   5.5903e-48   0.000000
  evidence P(D) = 7.4278e-07
  P(mutant | D) = P(AT|D) + P(TT|D) = 0.372249
```

6(a) 4 A + 1 T out of 5 reads; priors AT = 2 %, TT = 0.1 %, AA = 97.9 %

G	likelihood $P(D\|G)$	prior	likelihood × prior	posterior
AA	$C(5,1)(0.01)(0.99)^4 = 0.04803$	0.979	0.04702	**0.9377**
AT	$C(5,1)(0.5)^5 = 0.15625$	0.02	0.003125	**0.0623**
TT	$C(5,1)(0.99)(0.01)^4 = 4.95\times10^{-8}$	0.001	4.95×10^{-11}	$\approx 9.9\times10^{-10}$

Evidence P(D) = 0.05015. **P(mutant) = P(AT) + P(TT) ≈ 0.0623 (6.2 %)**: almost all of that is heterozygous, and P(TT) is about 10^{-9}. One T in five reads is three times more likely under AT than under AA (0.156 vs 0.048). But AA is 49× more likely a priori, so the single T is best explained as a sequencing error. The patient is most likely **AA** (93.8 %).

6(b) 24 A + 6 T out of 30 reads, same priors

G	likelihood	likelihood × prior	posterior
AA	$C(30,6)(0.01)^6(0.99)^{24} = 4.665\times10^{-7}$	4.567×10^{-7}	0.0397
AT	$C(30,6)(0.5)^{30} = 5.530\times10^{-4}$	1.106×10^{-5}	**0.9603**
TT	$C(30,6)(0.99)^6(0.01)^{24} = 5.59\times10^{-43}$	5.59×10^{-46}	$\approx 5\times10^{-41}$

P(mutant) ≈ 0.960 (96.0 %), essentially all of it heterozygous (AT). With 30 reads, six T's are very unlikely to come from 1% errors: we would expect only 0.3 erroneous T's. The likelihood ratio AT:AA is now about 1,185. That is enough to outweigh the 49:1 prior in favour of AA.

6(c) Same 24 A + 6 T, but a rarely-mutated site: AT = 0.05 %, TT = 0.001 %, AA = 99.949 %

G	likelihood	prior	likelihood × prior	posterior
AA	4.665×10^{-7}	0.99949	4.663×10^{-7}	0.6278
AT	5.530×10^{-4}	0.0005	2.765×10^{-7}	**0.3722**
TT	5.59×10^{-43}	0.00001	5.59×10^{-48}	$\approx 7.5\times10^{-42}$

P(mutant) ≈ 0.372 (37.2 %). The data are exactly the same as in (b), but the prior odds against a heterozygote are now about 2,000:1, not 49:1. That outweighs the likelihood ratio of about 1,185, so AA (error-only) is now the more probable explanation. The prior matters a lot for rare variants: the same read evidence can support a confident variant call at one site and a likely error at another. To call a variant here with confidence we would need more depth or higher-quality reads.